

Algorithms, Python, and Jupyter

Foundations, Functions, Debugging, and Profiling

DAEN 300

Alexander Abuabara

Fall 2026



Learning objectives

- Explain basic algorithms using precise pseudocode.
- Define and use Python functions.
- Use Jupyter/IPython tools to investigate errors.
- Read and interpret Python tracebacks.
- Measure execution time using `%time` and `%timeit`.
- Use profiling tools to identify performance bottlenecks.

The semester goal: explain programs to people

Let us change our traditional attitude to the construction of programs. Instead of imagining that our main task is to instruct a computer what to do, let us concentrate rather on explaining to human beings what we want a computer to do.

Donald Knuth, *Literate Programming*

- Code is read more often than it is written.
- Good programs explain ideas, not just instructions.
- Names, structure, and comments should guide the reader.
- The compiler is the second audience; humans are the first.

Part 1: Algorithms and Pseudocode

Before coding, make the logic precise

- Explain what an algorithm is and why precision matters.
- Read and write basic pseudocode using assignment, loops, and conditionals.
- Trace an algorithm by hand on a small input.
- Estimate running time using simple operation counts.
- Recognize common mistakes in algorithm design.

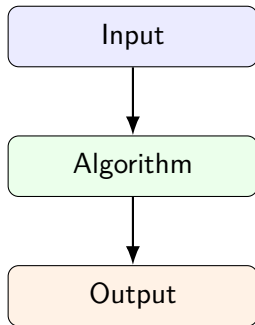
An algorithm transforms input into output

Working definition

An **algorithm** is a finite, precise sequence of steps that transforms input into output.

Good algorithms are:

- **Clear:** each step is unambiguous.
- **Finite:** the process eventually stops.
- **Correct:** it solves the stated problem.
- **Efficient:** it uses reasonable time and memory.



Pseudocode separates logic from syntax

Pseudocode describes logic without worrying about one programming language's syntax.

- Easier to discuss before coding.
- Focuses on decisions and repetition.
- Makes algorithms easier to compare.

Rule of thumb

Pseudocode should be detailed enough that a classmate can implement it without guessing the main idea.

Not the goal

Perfect syntax is less important than precise logic.

Basic pseudocode building blocks

```
x ← 0                // assignment

if score ≥ 70 then    // decision
    grade ← "pass"
else
    grade ← "fail"

for i ← 1 to n do     // counted repetition
    print(i)

while x < target do    // condition-based repetition
    x ← x + 1
```

- Use indentation to show which lines belong together.
- Choose meaningful variable names.
- State inputs and outputs when the algorithm is not obvious.

Example 1: Linear search

Problem

Given an array A and a value x , return the position of x if it appears. Otherwise return NOT-FOUND.

Algorithm LinearSearch(A, x)

Input: array $A[1..n]$, target value x

Output: index of x , **or** NOT-FOUND

```
for i <- 1 to n do
    if A[i] = x then
        return i
return NOT-FOUND
```

Tracing linear search

Suppose $A = [12, 7, 25, 4, 19]$ and $x = 4$.

Step	i	$A[i]$	Compare $A[i] = 4$?	Result	Continue?
1	1	12	no	–	yes
2	2	7	no	–	yes
3	3	25	no	–	yes
4	4	4	yes	return 4	no

Trace idea

A trace records how variables change while the algorithm runs.

Example 2: Binary search

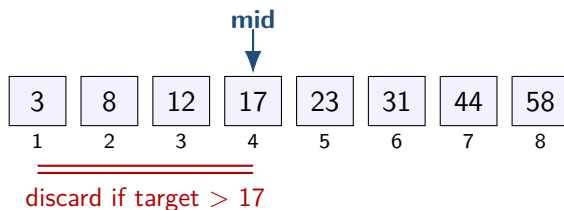
Key requirement

Binary search only works when the array is sorted.

```
Algorithm BinarySearch(A, x)
low <- 1
high <- n

while low <= high do
    mid <- floor((low + high) / 2)
    if A[mid] = x then
        return mid
    else if A[mid] < x then
        low <- mid + 1
    else
        high <- mid - 1
return NOT-FOUND
```

Binary search intuition



Each comparison removes about half of the remaining candidates.

Linear search: n candidates $\rightarrow$ Binary search: $\log_2 n$ rounds

Running time: the basic question

Instead of timing on one computer, we ask:

As input size grows,
how fast does the work grow?

Count the number of key operations as a
function of n .

- Linear search: at most n comparisons.
- Binary search: about $\log_2 n$ comparisons.
- Insertion sort: up to about $n^2/2$ shifts/comparisons.

Name	Big-O
Constant	$O(1)$
Logarithmic	$O(\log n)$
Linear	$O(n)$
Quadratic	$O(n^2)$

Correctness: what must be shown?

1. **Preconditions:** What must be true before the algorithm starts?
2. **Postconditions:** What must be true after it finishes?
3. **Progress:** Why does each loop move toward stopping?
4. **Invariant:** What useful fact stays true through each loop iteration?

Example

Binary search precondition: the input array is sorted. Without this, the algorithm may discard the wrong half.

Common pseudocode mistakes

Avoid:

- Using variables before defining them.
- Forgetting how indexes start: 0 or 1.
- Loops that never update the condition.
- Returning too early or too late.

Edge cases to test: empty input, one element, duplicate values, missing target, already sorted data.

Prefer:

- Clear input and output statements.
- Consistent indentation.
- Small examples to test edge cases.
- Comments for the main idea, not every line.

Mini-assignment

On a piece of paper, write your name or UIN and pseudocode for this task:

Find the maximum value

Given a non-empty array $A[1..n]$, return the largest value in the array.

Answer:

1. What variable stores the best value seen so far?
2. How many comparisons are made?
3. What happens when $n = 1$?

One possible solution

Algorithm Maximum(A)

Input: non-empty array $A[1..n]$

Output: largest value in A

```
best <- A[1]
```

```
for i <- 2 to n do
```

```
    if A[i] > best then
```

```
        best <- A[i]
```

```
return best
```

Running time

The loop performs $n - 1$ comparisons, so the algorithm is $O(n)$.

Algorithm design combines clarity, correctness, and efficiency

- Algorithms are precise recipes for transforming inputs into outputs.
- Pseudocode lets us focus on logic before programming syntax.
- Tracing helps catch mistakes and explain behavior.
- Big-O describes growth, not exact runtime on one machine.
- Correctness depends on assumptions, progress, and invariants.

Algorithm design = clarity + correctness + efficiency

Part 2: Python, Jupyter, and Functions

Jupyter supports an interactive development cycle

Jupyter provides an interactive environment for working with Python.

- Execute code one cell at a time.
- Immediately inspect results.
- Combine code, text, equations, and visualizations.
- Experiment without repeatedly running an entire program.
- Use special IPython “magic” commands.

Examples of magic commands:

`%run`

`%timeit`

`%prun`

`%xmode`

Check and install Python packages

Make sure a package is available before using it in Python.

In a terminal or command prompt:

```
# Show all installed packages  
pip list
```

```
# Check one package  
pip show pandas
```

```
# Install pandas if needed  
pip install pandas
```

In Jupyter Notebook:

```
%pip install pandas
```

If installation fails, check that Python and pip are using the same environment.

Load pandas and read a CSV file

Goal: Import pandas, load a CSV file, and preview the data.

```
# Load the pandas package
import pandas as pd

# Read a CSV file
df = pd.read_csv("data.csv")

# View the first five rows
df.head()
```

If the file is inside a folder:

```
df = pd.read_csv("data/myfile.csv")
```

Replace data.csv with your own file name.

Functions package reusable behavior

A function packages reusable code into a named block.

```
def square(x):  
    return x ** 2  
  
result = square(5)  
print(result)
```

Output:

```
25
```

Functions help make programs:

- reusable,
- easier to understand,
- easier to test,
- easier to debug.

Functions can accept multiple inputs

Functions can accept multiple arguments.

```
def average(a, b):  
    return (a + b) / 2  
  
print(average(10, 20))
```

Output:

```
15.0
```

A good function should generally perform one clearly defined task.

Functions reduce repetition

Without a function:

```
result1 = 5 ** 2  
result2 = 8 ** 2  
result3 = 12 ** 2
```

With a function:

```
def square(x):  
    return x ** 2  
  
result1 = square(5)  
result2 = square(8)  
result3 = square(12)
```

The second version is easier to reuse and modify.

Running Python Files with %run

IPython allows Python scripts to be executed directly.

```
%run my_script.py
```

If `my_script.py` defines functions or variables, they can then be available in the interactive IPython session.

This is useful when:

- developing functions in separate Python files,
- testing scripts interactively,
- combining scripts with notebook analysis.

Part 3: Debugging and Profiling

Debugging turns failures into evidence

Debugging is the process of finding and correcting errors in a program.

Common Python errors include:

- `SyntaxError`
- `NameError`
- `TypeError`
- `IndexError`
- `KeyError`
- `ZeroDivisionError`

Python error messages provide information about where and why the program failed.

Example: A Bug

Consider the following function:

```
def divide(a, b):  
    return a / b
```

```
divide(10, 0)
```

Python produces a traceback ending with:

```
ZeroDivisionError: division by zero
```

The traceback tells us:

1. what type of error occurred,
2. where it occurred,
3. which function calls led to it.

Read a traceback from the bottom upward

A useful strategy is to read a traceback from the bottom upward.

1. Identify the exception type.
2. Read the error message.
3. Find the line where the error occurred.
4. Follow the function-call chain if necessary.
5. Inspect the values involved.

The final line often provides the most important clue.

IPython Traceback Modes

IPython provides the `%xmode` magic command.

```
%xmode Plain  
%xmode Context  
%xmode Verbose
```

The modes control how much information appears when an exception occurs.

- **Plain:** minimal traceback.
- **Context:** includes surrounding context.
- **Verbose:** displays additional variable information.

Interactive Debugging

IPython also supports interactive debugging.
After an exception:

```
%debug
```

This allows you to inspect the program state near the point of failure.
Useful debugger commands can include:

- inspecting variable values,
- moving through the call stack,
- identifying which inputs caused the error.

Debugger Commands

ipdb> is the debugger prompt.

At that prompt, you can use commands such as:

```
n      # next line
s      # step into function
c      # continue execution
p x    # print x
pp x   # pretty-print x
u      # move up one stack frame
d      # move down one stack frame
q      # quit debugger
```

Profiling shows where computation is expensive

A program may be correct but still be slow.

Profiling means measuring where a program spends its time or memory.

The goal is to answer questions such as:

- Which function is slow?
- Which line takes the most time?
- Which operation uses the most memory?
- Where should optimization effort be focused?

Measure first; optimize second.

Timing with %time

The %time magic measures one execution.

```
%time sum(range(1000000))
```

It reports information such as:

- CPU time,
- system time,
- total execution time.

It is useful for obtaining a quick timing estimate.

Timing with %timeit

For more reliable measurements, use:

```
%timeit sum(range(1000000))
```

%timeit runs the statement repeatedly.

Advantages:

- reduces the effect of random timing variation,
- automatically chooses an appropriate number of repetitions,
- makes it easier to compare different approaches.

Line Magic vs. Cell Magic

A single percent sign applies to one line:

```
%timeit sum(range(1000))
```

Two percent signs apply to an entire cell:

```
%%timeit  
  
total = 0  
for i in range(1000):  
    total += i
```

Function Profiling with %prun

Suppose we have:

```
def calculate():  
    total = 0  
    for i in range(100000):  
        total += i ** 2  
    return total
```

Profile it with:

```
%prun calculate()
```

%prun reports which function calls consume the most time.

More Profiling Tools

The Python Data Science Handbook also introduces more detailed profiling tools.

`%lprun` Line-by-line execution profiling.

`%memit` Memory usage of a statement.

`%mprun` Line-by-line memory profiling.

These tools can help identify a specific line of code responsible for poor performance.

A reliable development workflow

A practical Jupyter development workflow:

1. Write a small function.
2. Run and test the function.
3. Inspect any traceback.
4. Debug the problem.
5. Confirm that the function is correct.
6. Measure its performance.
7. Profile slow sections.
8. Optimize only the important bottlenecks.

Functions, debugging, and profiling reinforce one another

- Functions organize Python code into reusable components.
- Jupyter makes experimentation and testing interactive.
- Tracebacks provide useful debugging information.
- IPython provides debugging tools such as `%xmode` and `%debug`.
- `%time` and `%timeit` measure execution time.
- Profiling reveals where optimization will have the greatest impact.



Jake VanderPlas, **Python Data Science Handbook**

<https://jakevdp.github.io/PythonDataScienceHandbook/01.00-ipython-beyond-normal-python.html>

Part 4: Assignment 1 (and Tips!)

Assignment 1

Submit your Jupyter Notebook on Canvas!

- Assignment 1 on Canvas
- Data: `events.csv`

Start by loading and inspecting the data

```
raw_df = pd.read_csv("events.csv")  
raw_df.head()
```

- `pd.read_csv()` reads a CSV file into a DataFrame.
- `.head()` previews the first five rows by default.
- Initial inspection reveals column names, sample values, and obvious formatting problems.

Inspect before converting types; raw data often violates expectations.

.copy() prevents accidental side effects

```
def standardize_text(df):  
    result = df.copy()  
    # modify result, not df  
    return result
```

Without a copy

- A function may mutate its input.
- Later cells can receive surprising data.

With a copy

- Input and output remain conceptually separate.
- Pipeline steps are easier to test and debug.

Normalize text with a chain of string methods

```
result[column] = (  
    result[column]  
        .astype("string")  
        .str.strip()  
        .str.lower()  
)
```

`.astype("string")` uses pandas' nullable string dtype; missing values remain missing.

`.str.strip()` removes leading and trailing whitespace.

`.str.lower()` converts letters to lowercase.

After normalization, values such as " Purchase " and "purchase" match.

Convert dates and numbers without stopping the pipeline

```
result["event_timestamp"] = pd.to_datetime(  
    result["event_timestamp"], errors="coerce"  
)  
result["quantity"] = pd.to_numeric(  
    result["quantity"], errors="coerce"  
)
```

- `pd.to_datetime()` parses dates and timestamps.
- `pd.to_numeric()` parses numeric values.
- `errors="coerce"` turns invalid input into `NaT` or `NaN` instead of raising an exception.

Coercion preserves the row and makes the quality problem measurable.

Use `.mask()` to invalidate values by a rule

```
result["quantity"] = result["quantity"].mask(  
    result["quantity"] <= 0  
)  
result["unit_price"] = result["unit_price"].mask(  
    result["unit_price"] < 0  
)
```

- `Series.mask(condition)` replaces values where the condition is `True`.
- With no replacement supplied, pandas inserts a missing value.
- The original row remains available for auditing.

A value can be syntactically numeric but still invalid under business rules.

Detect missing values and combine Boolean conditions

```
invalid_mask = (  
    clean_df["event_timestamp"].isna()  
    | clean_df["quantity"].isna()  
    | clean_df["unit_price"].isna()  
)  
invalid_rows = invalid_mask.sum()
```

- `.isna()` returns `True` for each missing value.
- `|` means elementwise OR; `&` means elementwise AND.
- Parenthesize every comparison in a compound pandas condition.
- Because `True == 1`, `.sum()` counts matching rows.

Remove exact duplicate rows

```
clean_df = result.drop_duplicates().copy()

removed = len(raw_df) - len(clean_df)
```

- `.drop_duplicates()` keeps the first occurrence of each exact row by default.
- `len(df)` returns the number of rows.
- A second `.copy()` makes the returned object independent and explicit.

Define what duplicate means; exact-row duplication is only one possible rule.

Select rows safely with `.loc`

```
purchases = result.loc[
    result["event_type"].eq("purchase")
    & result["country"].notna()
    & result["revenue"].notna()
]
```

- `.loc[row_condition]` selects rows by labels or a Boolean mask.
- `.eq("purchase")` is an explicit elementwise equality test.
- `.notna()` keeps values that are present.

Build the mask from named business requirements, then pass it to `.loc`.

Vectorized arithmetic computes an entire column

```
result["revenue"] = (  
    result["quantity"] * result["unit_price"]  
)
```

- pandas aligns the two Series by index and multiplies element by element.
- Missing quantity or price naturally produces missing revenue.
- No explicit Python loop is required.

Prefer column operations when the same calculation applies to every row.

.groupby() implements split-apply-combine

```
country_revenue = (  
    purchases  
    .groupby("country")["revenue"]  
    .sum()  
    .sort_index()  
)
```

1. **Split** rows into one group per country.
2. **Apply** `.sum()` to the revenue in each group.
3. **Combine** the results into a Series.
4. `.sort_index()` produces stable country ordering.

Filter first, group second, aggregate last.

The loop version exposes what vectorization replaces

```
totals = {}  
for row in df.itertuples(index=False):  
    revenue = row.quantity * row.unit_price  
    totals[row.country] = (  
        totals.get(row.country, 0) + revenue  
    )  
pd.Series(totals, dtype=float).sort_index()
```

- `.itertuples()` iterates through rows as named tuples.
- `dict.get(key, 0)` supplies zero for a country not yet seen.
- `pd.Series()` converts the dictionary back to pandas.

The loop is readable logic; the vectorized version delegates iteration to pandas.

Test correctness before comparing performance

```
pd.testing.assert_series_equal(  
    revenue_by_country_loop(clean_df),  
    revenue_by_country_vectorized(clean_df),  
    check_names=False,  
)  
  
%timeit revenue_by_country_vectorized(clean_df)  
%prun revenue_by_country_vectorized(clean_df)
```

- `assert_series_equal()` checks values, indexes, dtypes, and metadata.
- `%timeit` repeats execution to estimate runtime reliably.
- `%prun` profiles where one execution spends its time.

A faster answer matters only after it is shown to be the same answer.

Small summary functions turn quality into metrics

```
summary = {  
    "raw_rows": len(raw_df),  
    "unusable_timestamps": int(  
        clean_df["event_timestamp"].isna().sum()  
    ),  
    "total_valid_purchase_revenue": float(  
        purchase_revenue.sum()  
    ),  
}  
pd.Series(summary, name="value")
```

- `int()` and `float()` produce ordinary Python scalars.
- A dictionary gives each metric a clear name.
- `pd.Series(..., name="value")` creates a compact report.

Read the broken function as a debugging checklist

```
df[" revenue "] = df[" quantity "] * df[" unit_price "]
purchases = df[df[" event_type "] == " Purchase "]
return purchases.groupby(" country ")[ " reveneu "].sum()
```

- Column labels contain unintended spaces.
- The value " Purchase " was not normalized.
- "reveneu" is misspelled.
- The traceback identifies the failing expression; inspect exact labels before changing logic.

Cleaning values does not clean column names; exact spelling still matters.

A reliable pandas pattern: copy, transform, validate

1. Copy the input when a function should not mutate it.
2. Normalize text before testing categories.
3. Coerce malformed dates and numbers into measurable missing values.
4. Encode business rules with masks.
5. Filter with `.loc`, calculate by columns, and aggregate with `.groupby()`.
6. Assert equal results before timing or profiling.

One clear transformation per step makes the pipeline explainable.